

Ingeniería de Software II

Web Platform for Outfit Search
Project Proposal



Omar Nicolas Guerrero Guerrero
Jonathan Felipe López Núñez
Yeswah Gonzalez Tapia
Jose Alejandro Ordonez Bolanos
Nathalia Chaves Piarpuezan

Ingeniería de Sistemas y Computación
Universidad Nacional de Colombia
Bogotá DC Septiembre 2026

1. System Description

Shopping for clothes online can be a messy experience. If someone wants to build a full outfit, they usually have to open several store websites at the same time, compare prices by hand, keep track of sizes and colors in their head, and try to imagine whether the pieces will actually look good together. There is no single place that brings all of this together, so people end up spending a lot of time jumping between tabs and still end up unsure about their final choice.

Our service is a web application that solves this problem by putting the whole process in one place: discovering clothes, comparing them, and combining them into a full outfit. The platform collects clothing items from different online stores (using web scraping or public store data/APIs) and organizes them into a single, easy-to-browse catalog. Users can search for a specific item, and the app will show matching products from multiple stores side by side, comparing price, brand, material, color, available sizes, and store rating(as much as possible).

On top of the search feature, the app includes an outfit builder, where the user picks a top, bottom, shoes and can swap any single piece to instantly see how the whole outfit changes. Users can also set personal preferences (favorite brands, styles, colors, budget, fit, occasion, etc.).

2. Actors

End User (Shopper): the main actor of the system. Creates an account, sets style/budget preferences, searches for clothing items, builds and edits outfits, compares prices across stores, and follows a link out to the store to complete the purchase.

Partner Stores (external source): the online stores whose product data (name, price, images, sizes, stock) is collected through scraping or APIs. They are not direct users of the app, but the whole catalog depends on their data, and the final purchase always happens on their website.

3. Main Functionalities

- User registration/login and a preferences panel (favorite and disliked brands, preferred styles, colors, sizes, fit, occasion, season, and maximum budget), editable at any time.
- Multi-store product search with filters (price, brand, material, color, size, category, rating), pulling data from several stores into one results list.
- Detection and grouping of similar or equivalent products from different stores, so the user can compare prices for matching items.
- Visual outfit builder: choose a top, bottom, footwear, and accessories, replace any single piece, and see the outfit update in real time.
- Style-mixing option, letting the user combine two or more styles at once (e.g., Streetwear + Minimalist) to guide the recommendations shown.
- Outfit visualization screen showing the selected pieces together (as a flat-lay style composition, and ideally over a simple mannequin/avatar), so the user can judge the look before buying.
- Price comparison view for a chosen item or outfit, listing the price and store link for every matching option found.
- Saved outfits and favorites list, so users can come back to combinations they liked.

4. Members Roles

Omar Nicolas Guerrero Guerrero - Scrum Master

Jonathan Felipe Liriopez Nizioez - Developer

Yeswah Gonzalez Tapia - Developer

Jose Alejandro Ordonez Bolanos - Developer

Nathalia Chaves Piarpuezan - Developer

5. Technologies / Frameworks

Frontend: React.js (with Next.js) for the web interface, styled with Tailwind CSS, to build the search pages and the interactive outfit builder.

Backend: Node.js with Express to handle the API, user accounts, preferences, and the logic that talks to the scraping/data layer.

Database: MongoDB (or PostgreSQL or MySQL) to store users, preferences, saved outfits, and the cached product catalog.

Data collection: Python scripts (BeautifulSoup / Scrapy) or Puppeteer for web scraping store websites, run on a schedule to keep prices and stock up to date; public store APIs will be used instead of scraping wherever they are available.

Other tools: Git and GitHub for version control and teamwork, Figma for interface mockups before development, and Vercel or Render for deploying the web app.

Clarification and honesty note: Artificial intelligence was used to refine the idea and improve the syntax of the text.